Q10. Intelligent vs Creative Agent and Search, and Expanding the Action Space with RL

> **Question.** Explain intelligent agent and creative agent, intelligent search and creative search. If a creative agent must expand its search space and action space, how can we write a function for it? Take product design with a utility-based agent and reinforcement learning to expand the action space. Explain different options.

1. The idea in plain words

An **intelligent** agent is very good at reaching a goal with the moves it already has. A **creative** agent also asks: "*Do I have the right moves at all?*" If the answer is no, it invents new moves. In product design this means the agent does not only pick features one by one. It notices which feature *pairs* work well together and turns each pair into a single new action. The agent's action space grows, so the space of designs it can reach grows too.

2. Intelligent agent vs creative agent

	Intelligent agent	Creative agent
Aim	Rational: maximise the performance measure	Rational **but non-obvious**: also novel and useful
Structure	Agent = architecture + program	+ **creativity calibrator** and **conceptual space expander**
PEAS	Fixed P, E, A, S	Asks "what could be different?" for P, E, A and S
Actions	Fixed action set	Creates new actions (exploration, analogy, combination)
Learning	Improves its policy	The learn set evolves, so the algorithm evolves, so the outcome evolves

3. Intelligent search vs creative search

	Intelligent search	Creative search
Space	Search the given space efficiently	Create, discover or invent new options
Neighbours	Fixed neighbour definition	Redefines the neighbour
Nodes	Expands existing nodes	Combines two or more nodes (merging states)
Actions	Uses the action set as given	Creates new sets of actions
Goal	Fixed goal test	Goals may change (dynamic goals)
Result	Obvious best answer	Useful, surprising choices

4. Assumptions

- A phone has 6 slots (screen, battery, camera, body, sensors, AI) with 4 options each.
- **Basic action** = fix one slot to one option, giving 6 × 4 = **24 actions**. The agent has only **3 steps** per design.
- **Utility** = sum of feature quality + hidden **synergy bonuses** for 3 feature pairs (for example the right body with the right screen). The agent does not know the pairs. It must discover them.

- The agent is a utility-based learner using epsilon-greedy Q-learning. It runs 1,500 design episodes and expands its actions at episode 500.

5. A function that expands the search and action space

The idea: look at the best designs built so far and find **action pairs that keep appearing**. Each pair becomes a new macro-action that sets two slots in one step. With 3 steps, the agent can now fill up to 6 slots, so designs that were unreachable become reachable.

```
function expand(actions A, best_runs R):
    pairs ← count every consecutive pair (a_i, a_{i+1}) in the top runs R
    for each pair (a, b) among the top-k most frequent:
        if (a, b) not in A:  A.add( macro(a, b) )          # new action = a then b in one step
    return A                                                  # bigger action space, bigger
search space

function CreativeRL(A, episodes, warmup):
    for ep = 1 to episodes:
        if ep = warmup + 1: A ← expand(A, top 60 runs);  reset Q
        design ← run epsilon-greedy episode using actions in A
        reward ← utility(design)
        update Q(state, action) toward reward
    return Q, A
```

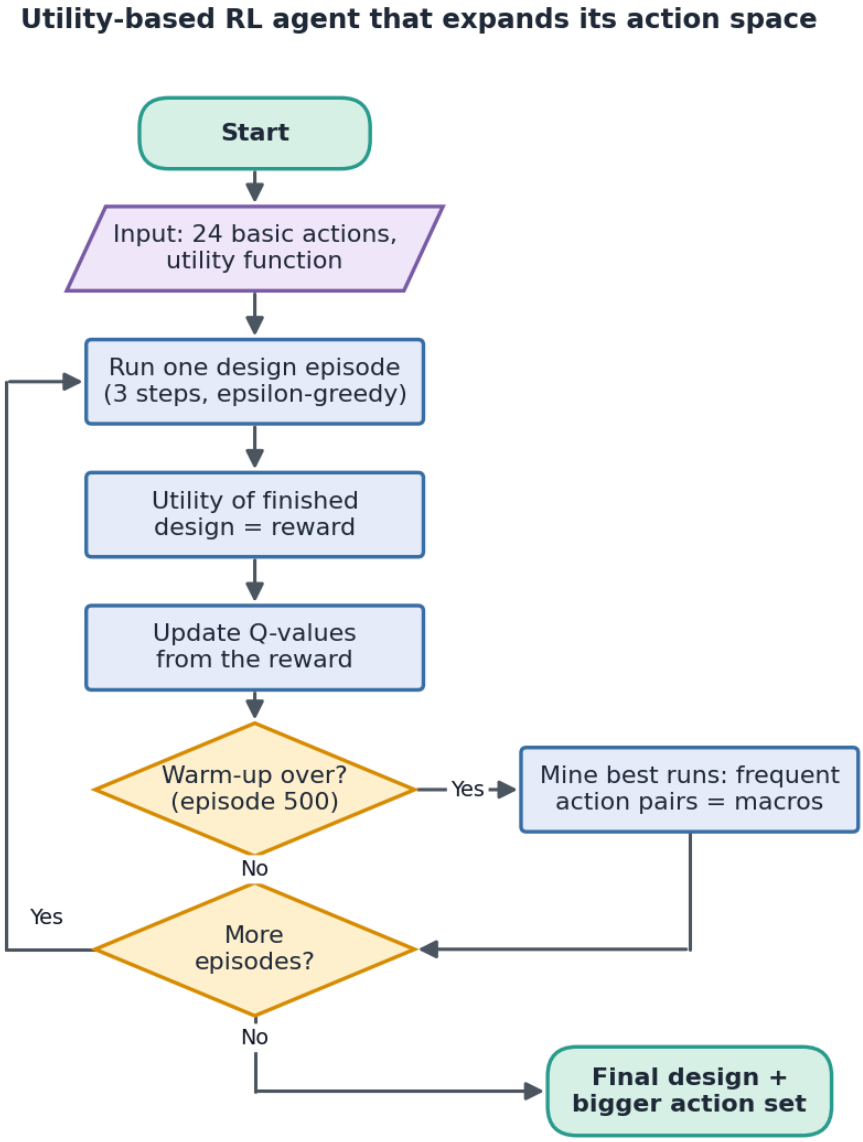


Figure 1. Flowchart of the utility-based RL agent that expands its action space.

6. Step-by-step approach

1. Start with 24 basic actions and the utility function (the agent's "happiness" measure).
2. Run design episodes. Each has 3 steps. The agent picks mostly the best-known action and sometimes a random one.
3. Score the finished phone with the utility function and update the Q-values with that reward.
4. After the warm-up (episode 500), look at the 60 best runs and mine the most frequent action pairs.
5. Add them as macro-actions. Clear the old Q-values, because the action list has changed.
6. Keep learning with the bigger action set. Compare with an agent whose actions never grow.

7. Worked example (real run)

The agent found these four macro-actions, which raised the action space from **24 to 28**:

New macro-action	Is it a hidden synergy?
body = 2 + screen = 1	Yes (bonus +2.0)
battery = 2 + sensors = 1	Yes (bonus +2.5)
screen = 1 + ai = 2	No, a lucky pair
body = 2 + battery = 2	No, a lucky pair

Two of the four were real synergies the agent had never been told about. Utility before and after the expansion:

Episodes	Fixed 24 actions	With expansion
1–500 (same for both)	1.92	1.92
501–1000	2.92	3.13
1001–1500	2.98	4.68
Final greedy design	**3.11**	**6.46**

The fixed agent stalls near 3 because three single-feature steps cannot reach the synergies. The creative agent ends **about 2 times higher** (6.46 vs 3.11) with the same three steps. Note the short dip just after episode 500 in Figure 2: we clear the Q-table when the actions change, so the agent has to relearn. The extra learning cost pays off later.

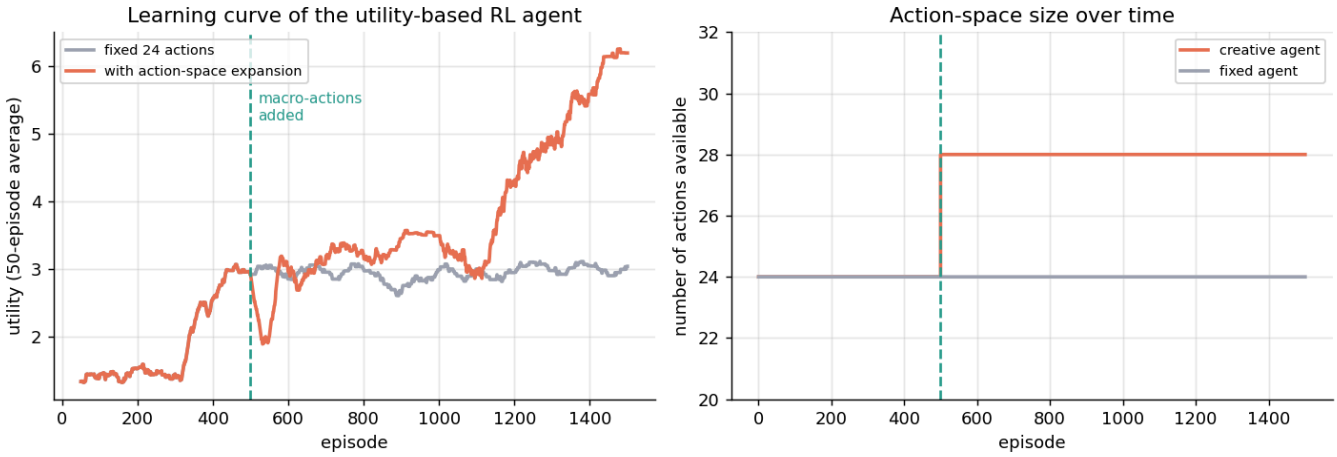


Figure 2. Left: utility per episode (50-episode average). Right: number of actions over time.

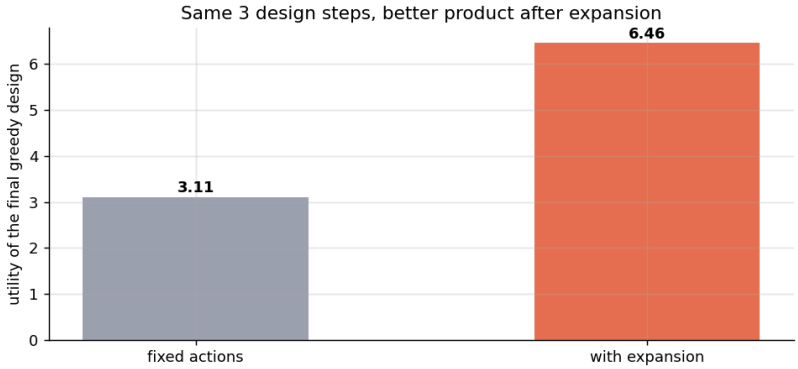


Figure 3. Utility of the final greedy design.

8. Different options to expand the action space

Option	What it does	Product design example
Macro-actions (options)	Turn frequent successful sequences into one action. *Used in our run.*	"Fold-able body + flexible screen" as one step
Parameterised actions	Make an action continuous instead of a fixed choice	Battery size from 3,000 to 6,000 mAh
Analogy / transfer	Copy actions that worked in another domain	Camera stabiliser ideas from drones
Human co-creator	A designer suggests a new action; the agent tests it	"Add a solar back panel"
Curiosity reward	Give extra reward for reaching new, unseen states	Tries rarely used feature mixes
GA over action sequences	Evolve sequences with crossover and mutation, keep the fittest as new actions	Mix two good build orders
New sensor / modality (IoT)	A new sensor adds a dimension to the space	Add a gas or health sensor slot

9. Code and real run

`q10_code.py` trains both agents, mines the macro-actions and saves the charts.

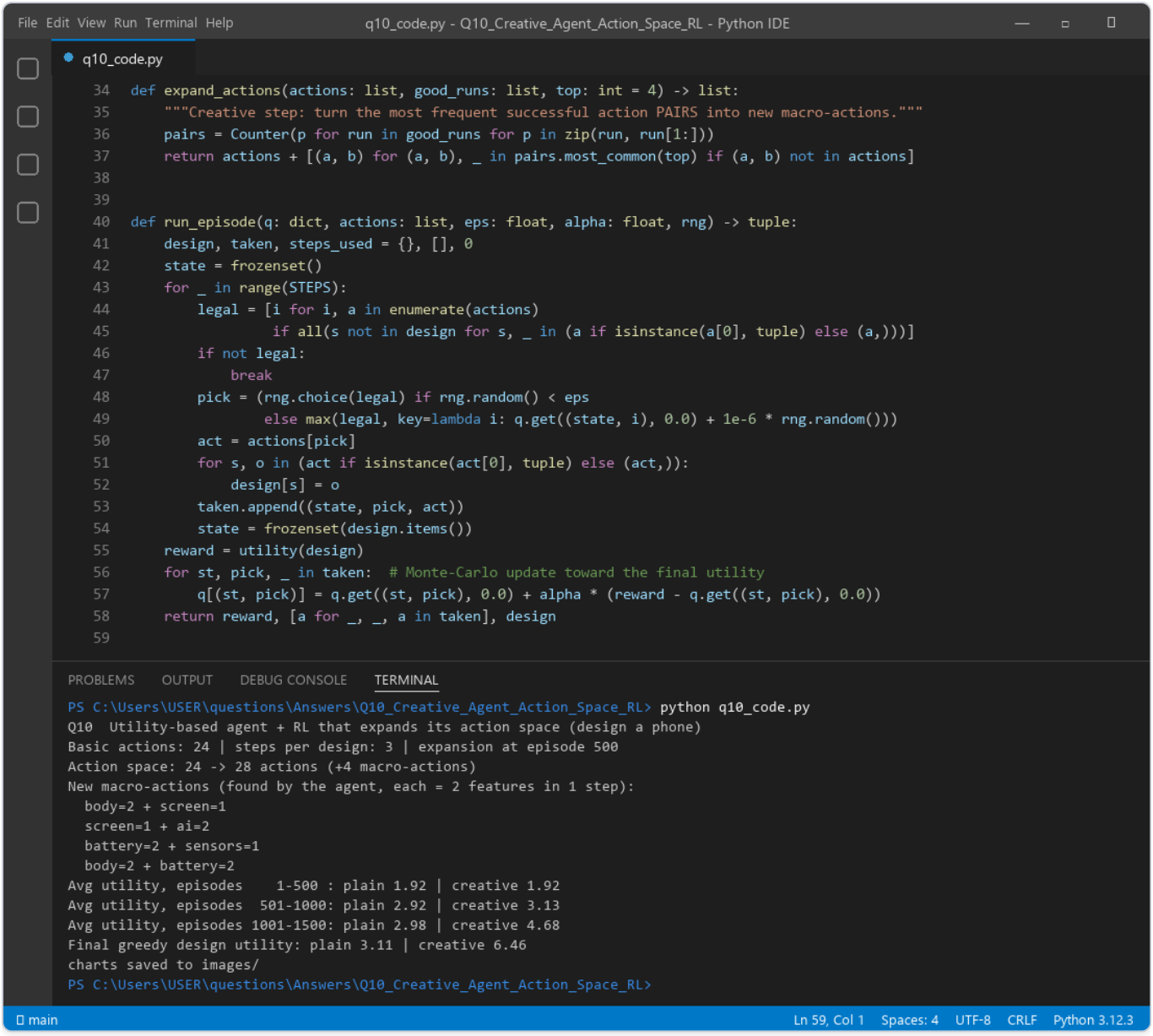


Figure 4. The `expand_actions()` and `run_episode()` functions with the real terminal output.

10. Summary

> An intelligent agent searches well inside the space it is given. A creative agent also changes the space: it creates new actions and redefines neighbours. By mining its best action pairs into macro-actions, the RL agent grew its action set from 24 to 28 and nearly doubled its final utility (3.11 to 6.46) in the same three steps. Other options are parameterised actions, analogy, human co-creators, curiosity and genetic search over action sequences.

Files in this folder: `q10_code.py` (source), `q10_code.ipynb` (Colab/Jupyter version, already run), `q10_output.txt` (console output), `images/` (all figures).